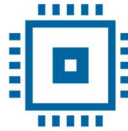




Transilvania
University
of Brasov



Transilvania
University
of Brasov
FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers
Study programme: Applied Electronics

Verification Project

Railway Management

Table of Contents

1	The digital design of the circuit	3
1.1	BLOCK DIAGRAM	3
1.2	DESCRIPTION OF THE GENERAL FUNCTIONALITY OF THE DUT (DEVICE UNDER TEST)	3
1.3	DUT INTERFACES	4
1.4	LIST OF DUT FUNCTIONALITIES	5
1.5	WAVEFORMS	6
2	Digital Circuit Verification	7
2.1	VERIFICATION ENVIRONMENT ARCHITECTURE	7
2.1.1	Trains Agent	7
2.1.2	Trains Transaction	7
2.1.3	Trains Driver	8
2.1.4	Trains Monitor	8
2.1.5	Trains Coverage Collector	8
2.1.6	Trains Sequences	8
2.1.7	Semaphore Agent	9
2.1.8	Semaphore Transaction	9
2.1.9	Semaphore Monitor	9
2.1.10	Semaphore Coverage Collector	9
2.1.11	Scoreboard	9
2.1.12	Scoreboard Coverage Collector	10
2.2	FUNCTIONAL COVERAGE ELEMENTS	10
2.3	VERIFICATION SCENARIOS AND TESTS	11
2.4	RESULTS	12

1 THE DIGITAL DESIGN OF THE CIRCUIT

Block Diagram

General Functionality Description of the DUT (Device Under Test)

DUT Interfaces

List of DUT Functionalities

Schematic Diagram

Waveforms

Register Table

1.1 BLOCK DIAGRAM

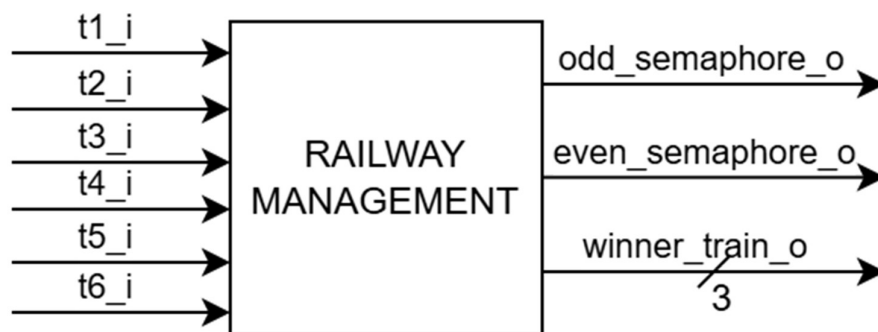


Figure 1. Block diagram of the Railway Management

1.2 DESCRIPTION OF THE GENERAL FUNCTIONALITY OF THE DUT (DEVICE UNDER TEST)

The Device Under Test (DUT) is a railway traffic controller designed to manage the access of six tourist trains to a shared railway section located near the red panda enclosure, which represents the central attraction of the zoo. Since all trains must pass through this section during their route, a control mechanism is required to prevent collisions and ensure safe operation.

The railway network is divided into two traffic directions. Trains T1, T3 and T5 approach the shared section from the west side, while trains T2, T4 and T6 approach it from the east side. To regulate access, a semaphore is installed at each end of the railway section. The DUT controls these semaphores and determines which train is allowed to enter the shared track segment.

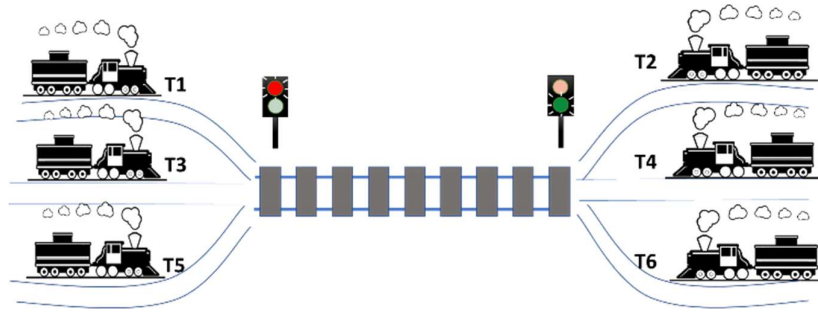


Figure 2. General idea of the railway access management controller

The controller operates according to a predefined set of priority rules. First, odd-numbered trains have higher priority than even-numbered trains. Then, trains with a smaller number have priority over trains with bigger number of the same parity. The railway section must remain unoccupied for one clock cycle between the passage of two trains belonging to different parity groups.

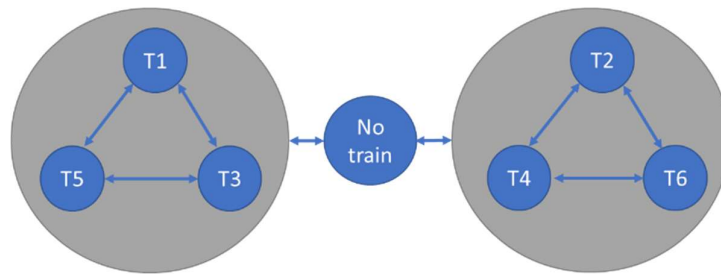


Figure 3. Finite State Machine (FSM) correlated with trains priority rules

The DUT continuously monitors the requests received from all trains and grants access to a single train at a time according to the established priority scheme. Based on the selected train, the corresponding semaphore is activated while the opposite semaphore remains blocked.

1.3 DUT INTERFACES

Table 1. System signals generated in the testbench

General Signals			
Signal Name	I/O	Size	Description
clk_i	I	1	Clock signal
reset_i	I	1	Asynchronous reset, active high

Table 2. Trains Interface

Trains Interface			
Signal Name	I/O	Size	Description
t1_i	I	1	Access request from train T1
t2_i	I	1	Access request from train T2
t3_i	I	1	Access request from train T3
t4_i	I	1	Access request from train T4
t5_i	I	1	Access request from train T5
t6_i	I	1	Access request from train T6

Table 3. Semaphore Interface

Semaphore Interface			
Signal Name	I/O	Size	Description
odd_semaphore_o	O	1	Semaphore control signal for odd-numbered trains
even_semaphore_o	O	1	Semaphore control signal for even-numbered trains
winner_train_o	O	3	Command for train that has access to the railway

1.4 LIST OF DUT FUNCTIONALITIES

- Continuously monitors request signals received from all six trains.
- Selects a single train to access the shared railway section based on the implemented priority rules.
- Controls the two railway semaphores and grants access only to the selected train.
- Ensures that the railway section remains unused for one clock cycle when switching between odd and even train groups.
- Prevents the same train from being granted access during two consecutive arbitration cycles.

1.5 WAVEFORMS

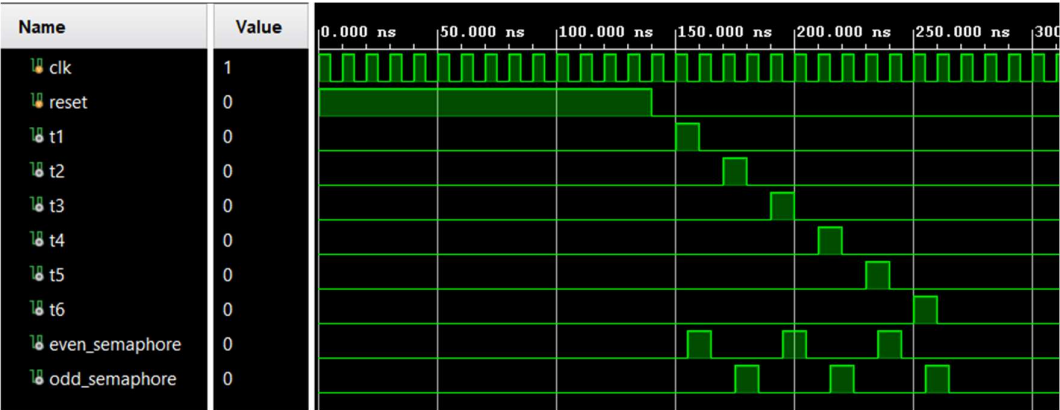


Figure 4. Waveforms from DUT for single train requests

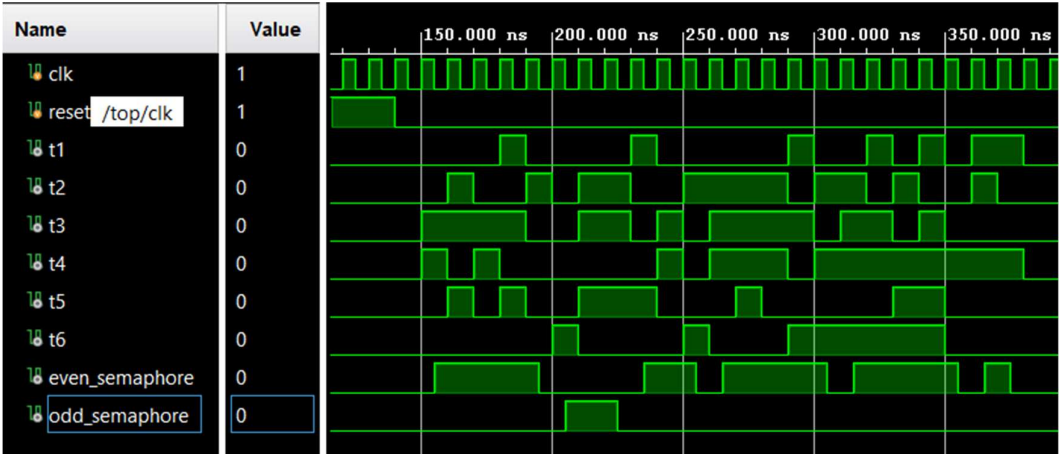


Figure 5. Waveforms from DUT for random train requests

2 DIGITAL CIRCUIT VERIFICATION

Verification Environment Architecture

Verification Checklist

Functional Coverage Elements

Verification Scenarios

Tests Results

2.1 VERIFICATION ENVIRONMENT ARCHITECTURE

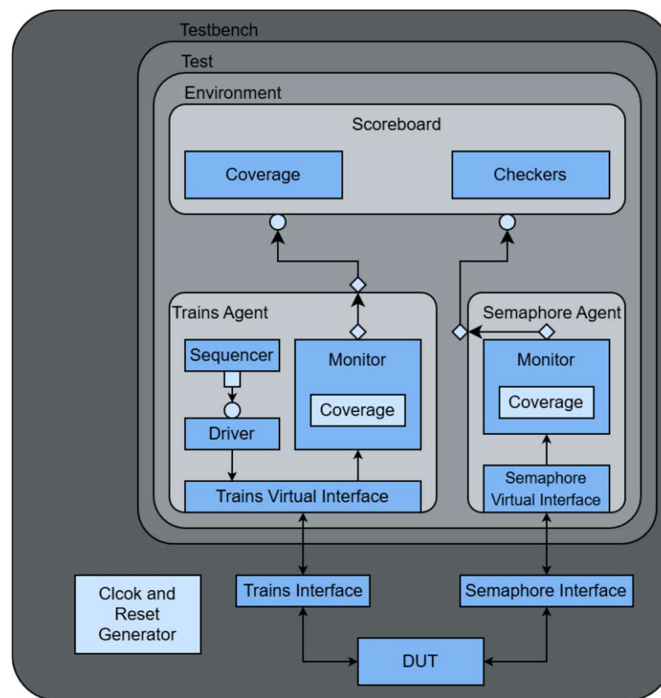


Figure 6. Overview of the components necessary for sending stimuli to the DUT

2.1.1 Trains Agent

The Trains Agent is an active agent that encapsulates the Sequencer, Driver and Monitor into a single entity, connecting the components with the DUT using the virtual interface.

2.1.2 Trains Transaction

Table 4. Trains Transaction

Field Name	Type	Size	Values	Description
t1_i	rand bit	1	[0:1]	Access request for T1
t2_i	rand bit	1	[0:1]	Access request for T2

t3_i	rand bit	1	[0:1]	Access request for T3
t4_i	rand bit	1	[0:1]	Access request for T4
t5_i	rand bit	1	[0:1]	Access request for T5
t6_i	rand bit	1	[0:1]	Access request for T6

2.1.3 Trains Driver

The Trains Driver interacts with the DUT via the Trains Interface, sending the transactions generated by the Trains Sequencer.

The driver receives transactions from the sequencer and drives the trains requests signals.

2.1.4 Trains Monitor

The Trains Monitor samples the request signals of the trains from Train Virtual Interface every clock cycle, reconstructs the transaction and sends it to the Scoreboard and Train Coverage Collector.

2.1.5 Trains Coverage Collector

The Trains Coverage Collector is instantiated inside the Trains Monitor and receives transactions from it. The coverage collector contains one coverpoint for each train request: t1, t2, t3, t4, t5, t6; and requests_cross.

2.1.6 Trains Sequences

The Trains Sequences generate stimulus for the Trains Driver. Each sequence creates one or more constrained-random transactions that the driver sends to Trains Interface. The following sequences are implemented:

Table 5. Trains Sequences

Sequence	Description
single_train_request_sequence	Sends requests from each train individually.
odd_even_priority_sequence	Sends simultaneous requests from one odd-numbered train and one even-numbered train.
same_parity_priority_sequence	Sends simultaneous requests from two trains belonging to the same parity group.
same_train_repeated_sequence	Sends repeated requests from the same train in two consecutive cycles.

all_requests_sequence	Sends requests from all trains simultaneously.
stress_sequence	Sends train requests at a higher rate than normal operating conditions.
trains_sequence	Sends random combinations of train requests.

2.1.7 Semaphore Agent

The Semaphore Agent is a passive agent that contains only the Monitor and connects it with the DUT using the virtual interface. The Semaphore Interface has only output signals from DUT, meaning no driver or sequencer is needed.

2.1.8 Semaphore Transaction

Table 6. Semaphore Transaction

Field Name	Type	Size	Values	Description
even_semaphore_state	rand bit	1	[0:1]	State of semaphore for even numbered trains
odd_semaphore_state	rand bit	1	[0:1]	State of semaphore for odd numbered trains
winner_train	rand train_state_t	3	{NO_TRAIN, TRAIN1, TRAIN2, TRAIN3, TRAIN4, TRAIN5, TRAIN6}	Train that has access to the railway

2.1.9 Semaphore Monitor

The Semaphore Monitor samples the semaphore states and the chosen train every clock cycle, reconstructs the transaction and sends it to the Scoreboard and Semaphore Coverage Collector.

2.1.10 Semaphore Coverage Collector

The Semaphore Coverage Collector is instantiated inside the Semaphore Monitor and receives transactions from it. The coverage collector contains 3 cover points: even_semaphore_state_cp, odd_semaphore_state_cp and winner_train_cp.

2.1.11 Scoreboard

The Scoreboard receives transactions from both monitors.

When receiving a Trains Transaction, the scoreboard predicts the next train that will enter the railway and the next activated semaphore, meaning the next Semaphore Transaction.

All predicted Semaphore Transactions are stored in a queue.

When receiving a Semaphore Transaction, the scoreboard compares it with the predicted transaction, generating an error message if the checker fails.

2.1.12 Scoreboard Coverage Collector

The Scoreboard Coverage Collector is instantiated inside the Scoreboard and receives transactions from it. The coverage collector contains 2 coverpoints: `current_state` and `next_state`; and transitions `cross`.

2.2 FUNCTIONAL COVERAGE ELEMENTS

Table 7. Trains cover group

trains_requests_cg		
Cover point	Values	Description
t1	[0:1]	Covers request of T1
t2	[0:1]	Covers request of T2
t3	[0:1]	Covers request of T3
t4	[0:1]	Covers request of T4
t5	[0:1]	Covers request of T5
t6	[0:1]	Covers request of T6
requests_cross	t1 x t2 x t3 x t4 x t5 x t6	Covers all combinations of requests

Table 8. Semaphore cover group

semaphore_states_cg		
Cover point	Values	Description
even_semaphore_state_cp	{GREEN, RED}	Covers states of semaphore for even_numbered trains
odd_semaphore_state_cp	{GREEN, RED}	Covers states of semaphore for odd_numbered trains
winner_train_cp	{NO_TRAIN, TRAIN1, TRAIN2, TRAIN3, TRAIN4, TRAIN5, TRAIN6}	Covers the trains that have access to the railway

Table 9. Scoreboard cover group

fsm_cg		
Cover point	Values	Description
current_state	{NO_TRAIN, TRAIN1, TRAIN2, TRAIN3, TRAIN4, TRAIN5, TRAIN6}	Covers current state of the railway
next_state	{NO_TRAIN, TRAIN1, TRAIN2, TRAIN3, TRAIN4, TRAIN5, TRAIN6}	Covers next predicted state of the railway
state_transition	current_state x next_state	Covers the valid transitions between railway states

2.3 VERIFICATION SCENARIOS AND TESTS

Table 10. Verification Scenarios

Scenario	Description
Single Train Request Verification	Individual train requests are applied to verify correct semaphore activation and train selection.
Odd-Even Priority Verification	Simultaneous requests from odd-numbered and even-numbered trains are applied to verify the priority rules.
Same Parity Priority Verification	Simultaneous requests from trains belonging to the same parity group are applied to verify the internal priority hierarchy.
All Requests Verification	Requests from all trains are asserted simultaneously to verify the complete arbitration mechanism.
Repeated Train Request Verification	Consecutive requests from the same train are applied to verify that the train cannot be selected in consecutive arbitration cycles.
Idle Cycle Verification	Request sequences are applied to verify insertion of the idle state when switching between odd and even train groups.
Stress Traffic Verification	Frequent train requests are generated to evaluate DUT behavior under heavy traffic conditions.

Table 11. List of implemented tests for DUT verification

Test file name	Test description
single_train_request_test	Test that DUT grants access correctly when requests are generated by a single train.
odd_even_priority_test	Test that DUT prioritizes odd-numbered trains over even-numbered trains.
same_parity_priority_test	Test that DUT applies the correct priority order between trains of the same parity.
same_train_repeated_test	Test that DUT does not grant access to the same train in consecutive arbitration cycles.
all_requests_test	Test that DUT applies the complete priority hierarchy when all trains request access simultaneously.
stress_test	Test that DUT operates correctly under heavy traffic conditions with frequent train requests.
trains_test	Test that DUT handles random train requests correctly.

2.4 RESULTS

Running the regression produced the following results:

- Trains functional coverage: 100%
- Semaphore functional coverage: 100%
- Scoreboard coverage: 100%

Semaphore assertions passed without failures throughout the simulation